Defining and using structures

A structure in C is a user-defined data type that groups related variables of different data types under a single name.

Defining a Structure

```
struct Student {
    int id;
    char name[20];
    float mark;
};
```

Using a Structure/Access Structure Members

```
#include <stdio.h>

// Defining a structure
struct A {
    int x;
};

int main() {

    // Creating a structure variable
    struct A a;

        // Initializing member
        a.x = 11;

    printf("%d", a.x);
    return 0;
}
```

11

```
#include <stdio.h>

struct Student {
    int id;
    char name[20];
    float mark;
};

int main() {
    struct Student s1;

    s1.id = 101;
    s1.mark = 85.5;

    printf("Enter name: ");
    scanf("%19s", s1.name);

    printf("ID: %d\n", s1.id);
```

```
    printf("Name: %s\n", s1.name);
    printf("Mark: %.2f\n", s1.mark);
    return 0;
}
```
Enter name: Ravi
ID: 101
Name: Ravi
Mark: 85.50

Initializing a Structure Directly

```
#include <stdio.h>

struct Student {
    int id;
    char name[20];
    float mark;
};

int main() {
    struct Student s1 = {101, "Ravi", 85.5};

    printf("%d\n", s1.id);
    printf("%s\n", s1.name);
    printf("%.2f\n", s1.mark);

    return 0;
}
```
101
Ravi
85.50

Copy Structure

```
#include <stdio.h>
struct Student {
    int id;
    char grade;
};

int main() {
    struct Student s1 = {1, 'A'};

    // Create a copy of student s1
    struct Student s2 = s1;

    printf("Student 1 ID: %d\n", s2.id);
    printf("Student 1 Grade: %c", s2.grade);
    return 0;
}
```
Student 1 ID: 1
Student 1 Grade: A

Passing Structure to Functions

```
#include <stdio.h>

struct Student {
    int id;
    char name[20];
};

void display(struct Student s) {
    printf("Student ID: %d\n", s.id);
    printf("Student Name: %s\n", s.name);
}

int main() {
    struct Student s1 = {101, "Ravi"};

    display(s1);  // Passing structure to function

    return 0;
}
```

Student ID: 101
Student Name: Ravi

```
#include <stdio.h>

// Structure definition
struct A {
    int x;
};

// Function to increment values
void increment(struct A a1, struct A *b1) {
    a1.x=a1.x+1;
    b1->x++;
printf(“%d”,a1.x);
}

int main() {
    struct A a  = { 10 };
          struct A b  = { 10 };

          // Passing a by value and b by pointer
          increment(a, &b);

          printf("a.x: %d \tb.x: %d", a.x, b.x);
    return 0;
}
```

a.x: 10 b.x: 11

Size of a Structure

```
#include <stdio.h>

struct Student {
    int id;
    float mark;
};

int main() {
    struct Student s1;

    printf("Size of int: %zu bytes\n", sizeof(int));
    printf("Size of float: %zu bytes\n", sizeof(float));
    printf("Size of structure: %zu bytes\n", sizeof(s1));

    return 0;
}
```

Size of int: 4 bytes
Size of float: 4 bytes
Size of structure: 8 bytes

Nested Structures

```
#include <stdio.h>

// Child structure declaration
struct child {
    int x;
    char c;
};

// Parent structure declaration
struct parent {
    int a;
    struct child b;
};

int main() {
    struct parent p = { 25, 195, 'A' };

    // Accessing and printing nested members
    printf("p.a = %d\n", p.a);
    printf("p.b.x = %d\n", p.b.x);
    printf("p.b.c = %c", p.b.c);
    return 0;
}
```

p.a = 25
p.b.x = 195
p.b.c = A

```
#include <stdio.h>
```

```
struct Date {
    int day;
    int month;
    int year;
};

struct Student {
    int id;
    char name[20];
    struct Date dob;
};

int main() {
    struct Student s1 = {101, "Ravi", {15, 8, 2005}};

    printf("ID: %d\n", s1.id);
    printf("Name: %s\n", s1.name);
    printf("Date of Birth: %d-%d-%d\n",s1.dob.day, s1.dob.month, s1.dob.year);

    return 0;
}
```

ID: 101
Name: Ravi
Date of Birth: 15-8-2005

Array of structures

An array of structures is used to store details of multiple records of the same structure type.

```
#include <stdio.h>

struct Student {
    int id;
    float mark;
};

int main() {
    struct Student s[3] = {
        {101, 85.5},
        {102, 90.0},
        {103, 78.5}
    };

    int i;

    for (i = 0; i < 3; i++) {
        printf("ID: %d, Mark: %.1f\n",
               s[i].id, s[i].mark);
    }

    return 0;
```

```
}
ID: 101, Mark: 85.5
ID: 102, Mark: 90.0
ID: 103, Mark: 78.5
```

```
#include <stdio.h>

struct A {
    int var;
    char c;
};

int main() {
    struct A arr[2] = {
        {1, 'a'},
        {2, 'b'}
    };

    for (int i = 0; i < 2; i++) {
        printf("%d %c\n", arr[i].var, arr[i].c);
    }

    return 0;
}
1 a
2 b
```

Pointers to structures (Structure Pointer)

A pointer to a structure is a pointer that stores the address of a structure variable.

```
#include <stdio.h>

struct Point {
    int x, y;
};

int main() {
    struct Point p = { 1, 2 };

    struct Point *ptr = &p;

    printf("%d %d", ptr->x, ptr->y);

    return 0;
}
1 2
```

```
#include <stdio.h>
#include <string.h>

struct Student {
    int roll_no;
    char name[30];
```

```
    char branch[40];
    int batch;
};

int main() {

    struct Student s1 = {27, "Geek", "EEE", 2026};

    struct Student *ptr = &s1;

    printf("%d\n", ptr->roll_no);
    printf("%s\n", ptr->name);
    printf("%s\n", ptr->branch);
    printf("%d", ptr->batch);

    return 0;
}
```

27
Geek
EEE
2026

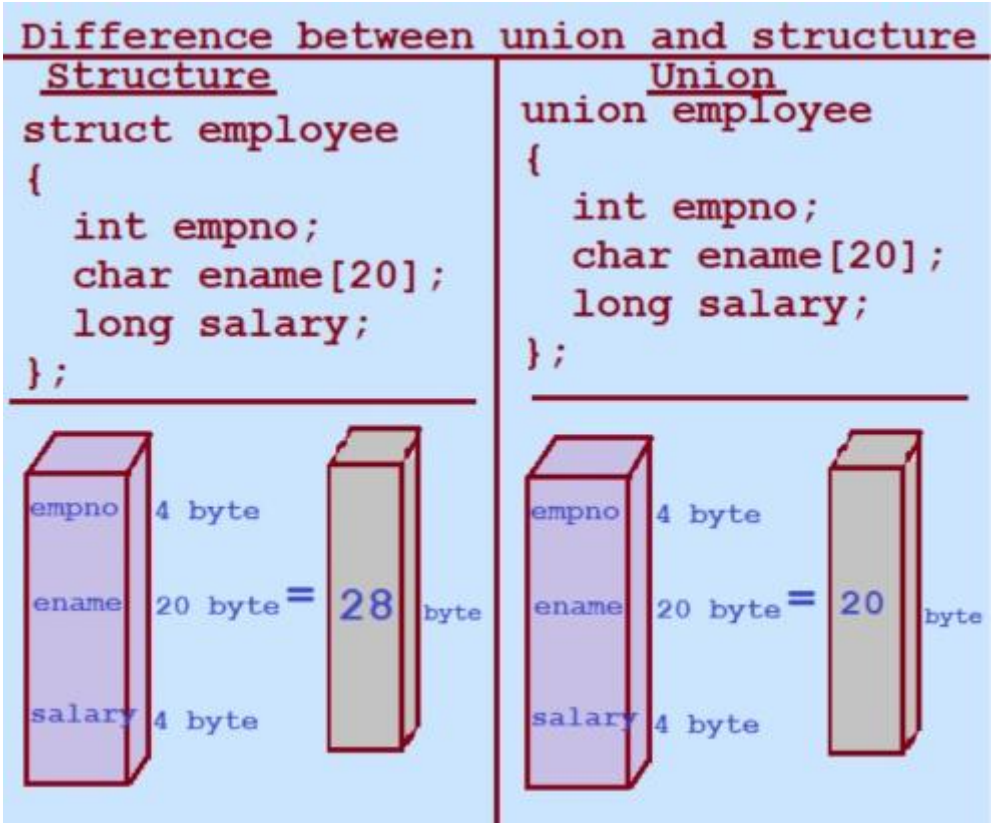
Unions and their uses

A union is a user-defined data type that allows storing different data types in the same memory location.
At any given time, only one member can hold a valid value because all members share the same memory.

Syntax

```
union Data {
    int i;
    float f;
    char c;
};
```

Here, i, f, and c share the same memory location.




```
#include <stdio.h>

union Data {
    int i;
    float f;
    char c;
};

int main() {
    union Data d;

    printf("Size of int: %zu bytes\n", sizeof(d.i));
    printf("Size of float: %zu bytes\n", sizeof(d.f));
    printf("Size of char: %zu bytes\n", sizeof(d.c));

    printf("Size of union: %zu bytes\n", sizeof(d));

    return 0;
}
Size of int: 4 bytes
Size of float: 4 bytes
Size of char: 1 bytes
Size of union: 4 bytes
```

Simple Union Program

```
#include <stdio.h>
```

```
union Data {
    int i;
    float f;
    char c;
};

int main() {
    union Data d;

    d.i = 10;
    printf("Integer: %d\n", d.i);

    d.f = 5.5;
    printf("Float: %.1f\n", d.f);

    d.c = 'A';
    printf("Character: %c\n", d.c);

    return 0;
}
```

```
Integer: 10
Float: 5.5
Character: A
```

d.i = 10; stores an integer.
d.f = 5.5; stores a float in the same shared memory.
d.c = 'A'; stores a character in the shared memory.

Nested Union

```
#include <stdio.h>

union A {
    int marks;
};

union B {
    int rollNo;
    union A p;
};

int main() {
    union B s;

    s.rollNo = 21;
    printf("Roll Number: %d\n", s.rollNo);

    s.p.marks = 91;
    printf("Marks: %d\n", s.p.marks);

    return 0;
}
```

Roll Number: 21
Marks: 91

Uses of Union

- **Memory saving:** Useful when only one of several data values is needed at a time.
- **Embedded systems:** Useful for memory-conscious applications.
- **Hardware/register handling:** Can be used in low-level programming with appropriate platform-specific care.
- **Variant data:** Useful for representing a value that can be one of several types.

Structure vs. Union

Feature	Structure	Union
Keyword	`struct`	`union`
Member access	All members can be used at once	Only one member at a time
Memory	Each member gets its own slot	All members share one slot
Size	Sum of all member sizes (plus padding)	Size of the largest member
Changing one member	Does not affect other members	Overwrites the others
Initialization	Multiple members can be initialized	Only one member can hold a value
Typical use	Records, entities, grouped data	Variant types, saving memory

Enumeration (enum) in C

An Enumeration (enum) is a user-defined data type in C that contains a set of named integer constants.
It makes programs easier to read and understand.

Syntax

```
enum week {
    MON, TUE, WED, THU, FRI, SAT, SUN
};
```

By default, enumeration values start from 0.

Simple Enumeration Program

```
#include <stdio.h>

enum week {
    MON, TUE, WED, THU, FRI
};

int main() {
    enum week today;

    today = WED;

    printf("Today is day number: %d", today);

    return 0;
}
```

Today is day number: 2

```
#include <stdio.h>

enum Level {
  LOW,
  MEDIUM,
  HIGH
};

int main() {

  enum Level myVar = MEDIUM;
   printf("%d", myVar);

  return 0;
}
```

1

Enumeration with User-Defined Values

```
#include <stdio.h>

enum week {
    MON = 1,
```

```
    TUE = 2,
    WED = 3,
    THU = 4,
    FRI = 5
};

int main() {
    enum week today;

    today = WED;

    printf("Day number: %d", today);

    return 0;
}
```
Day number: 3

```
#include <stdio.h>

enum Level {
  LOW = 25,
  MEDIUM = 50,
  HIGH = 75
};

int main() {
  enum Level myVar = MEDIUM;
  printf("%d", myVar);

  return 0;
}
```
50

```
#include <stdio.h>

enum Level {
  LOW = 5,
  MEDIUM,
  HIGH
};

int main() {
  enum Level myVar = MEDIUM;
  printf("%d", myVar);

  return 0;
}
```
6

Uses of Enumeration

1. Days: Represent days of the week.
2. Months: Represent months of the year.

3. Traffic signals: Represent RED, YELLOW and GREEN.
4. Menu options: Represent different menu choices.
5. Status: Represent SUCCESS, FAILURE and PENDING.